Lab Test 2 [25 marks]

Coding time: 1 hour and 30 mins

Instructions on how to download the resource files:

1. Before the test begins, you will be instructed to download the resource files from eLearn.

General Instructions:

1. You will take the lab test on your personal laptop. Remember to **REMOVE** all AI-based extensions and tools on your laptop before the test if any. **Using any AI-based tools during the test WILL be considered as cheating and subjected to disciplinary actions.**

2. You are not allowed to communicate with anyone or access any external resources (except for downloading/uploading files from/to eLearn) during the test. **Anyone caught doing so during the test WILL be considered as cheating and subjected to disciplinary actions.**

3. You may refer to any file on your laptop during the test.

4. Make sure your code can generate the same output as we show in the sample runs. You may be penalized for missing spaces, missing punctuation marks, misspellings, etc. in the output.

5. Do not hardcode. We will use different test cases to test and grade your solutions.

6. Python script files that cannot be executed will NOT be marked and hence you will be awarded 0 marks. You may wish to comment out the parts in your code which cause execution errors.

7. Include your name as author in the comments of all your submitted source files. For example, include the following block of comments at the beginning of each source file you need to submit.

```
# Name    : BAI She Jing
# Email ID: shejing.bai.2024
```

Instructions on how to submit your solutions:

1. When the test ends, zip up all the files required for submission in a zip archive. The name of the zip archive should be your email ID. For example, if your email is `shejing.bai.2024@scis.smu.edu.sg`, you should name the archive as **`shejing.bai.2024.zip`**. You may be penalized for not following our instructions.

2. Once everybody has zipped up his/her files, your invigilator will instruct you to submit your solutions as a single zip file to eLearn under Assignments.

Question 1 [8 marks]

Part A The Cargo Challenge (Difficulty Level: *)

You are helping a cargo delivery company that transports various types of goods across the globe. Each shipment is represented as a tuple containing two values:

- The weight of the item in kg (type: `float`)
- The dollar value of the item (type: `int`)

As a cost-saving measure, the company is only interested in the total value of shipments where the weight is less than or equal to 10.0 kg (since parcels of lighter weight can be transported with a special discount on fuel costs).

In `q1a.py`, write a function `sum_values_by_weights(lst_tups)` that takes one argument:

- `lst_tups` (type: `list`): A list of cargo shipments (each represented by a tuple of a float and an integer).

The function should return the sum of the dollar values of those shipments where the weight is less than or equal to 10.0 kg.

The function should return the integer 0 if no shipment weighs at or below 10.0 kg.
The function should return `None`, if given an empty list of cargo shipments.

Examples:

- `sum_values_by_weights([(9.9, 4), (11.25, 5), (10.0, 10), (18.70, 100)])` returns: `14` because only the first and third shipments have weights less than or equal to 10.0 kg, giving a total dollar value of 4 + 10 = 14.

- `sum_values_by_weights([(13.0, 4), (21.03, 8), (10.01, 50), (19.99, 29)])` returns: `0` because none of the shipments have a weight of 10.0 kg or less.

- `sum_values_by_weights([])` returns: `None` for an empty list of cargo shipments.

Use `q1a_test.py` to test your code. Your `q1a.py` should contain the entire function definition of `sum_values_by_weights`. We will only grade your `q1a.py` using a different test script.

Part B The Secret Message Generator (Difficulty Level: **)

You are part of an elite team of cryptographers tasked with sending secret messages between resistance groups fighting a tyrannical government in a dystopian future. To avoid detection, you need to create encrypted messages using two separate strings of characters. You've developed a method where the two strings are combined by choosing alternating characters between strings. However, when one string is longer than the other, you reverse the remaining characters of the longer string before appending them to the final message. This trick helps throw off enemy decryption attempts!

In `q1b.py`, write a function `alternate_and_reverse(s1, s2)` that takes two strings, possibly of different lengths, and returns a new encrypted string.

The function alternates between characters from `s1` and `s2`. If one string is longer than the other, append the extra characters from the longer string in reverse order at the end of the new string.

The function should return an empty string (`''`) if both input strings are empty. If only one of the input strings is empty, the function should return the other string in reverse order.

Examples:

- `alternate_and_reverse('', '')` returns `''`
- `alternate_and_reverse('', 'abcde')` returns `'edcba'`
- `alternate_and_reverse('xyz123', '')` returns `'321zyx'`
- `alternate_and_reverse('abc', 'def')` returns `'adbecf'`
- `alternate_and_reverse('abcde', '123')` returns `'a1b2c3ed'`
- `alternate_and_reverse('xyz', '78901')` returns `'x7y8z910'`

Use `q1b_test.py` to test your code. Your `q1b.py` should contain the entire function definition of `alternate_and_reverse`. We will only grade your `q1b.py` using a different test script.

Question 2 [8 marks]

- Assume all entries in the data file are valid.
- Assume all arguments given to the below functions are valid.
- Note: all strings are case-sensitive.

Part A (Difficulty Level: *)

Implement a function called `get_entry_count` in `q2a.py`. This function takes two parameters.

- `filename` (type: `str`): The name of a text file.
- `swimmer_name` (type: `str`): An optional parameter that is the name of a swimmer in `filename`. The default value is `None`.

We assume the text file contains at least one line. Each line contains a swimmer's name, the name of an event, and the swim time in seconds (integer). Each column is separated by the comma character (i.e. `","`).

For example, `swimtimes-sample.csv` has four entries, as shown below:

```
Michael Phelps,50m doggy paddle,60
Shanti Pereira,50m backstroke,42
Shanti Pereira,50m breaststroke,35
Michael Phelps,50m doggy paddle,65
```

If an argument is provided for `swimmer_name`, the function returns the number of entries (type: `int`) for the swimmer.

If no argument is provided for `swimmer_name`, the function returns the number of entries (type: `int`) that are in the file.

Examples:

- `get_entry_count("swimtimes-sample.csv")` returns `4`.
- `get_entry_count("swimtimes-sample.csv", "Shanti Pereira")` returns `2`.

Use `q2a_test.py` to test your code. Note that it also uses `swimtimes-sample2.csv` for some tests. Your `q2a.py` should contain the entire function definition of `get_entry_count`. We will grade only your `q2a.py` using a different test script.

Part B (Difficulty Level: **)

Implement a function called `get_swim_report` in `q2b.py`. This function takes three parameters.

- `filename` (type: `str`): The name of a text file, which is the same as in Part A.
- `swimmer_name` (type: `str`): The name of a swimmer.
- `event_name` (type: `str`): The name of an event.

The function returns a tuple `(count, average)`.

- `count` (type: `int`): The number of entries for the swimmer `swimmer_name` for the event specified by `event_name`.
- `average` (type: `int`): The average time for the swimmer in the specified event. `average` should be rounded to the nearest integer using the `round` function. (Hint: `round(3.14) => 3`)

The function should return `(0, 0)` if the swimmer does not have an entry for the event.

Examples:

- `get_swim_report("swimtimes-sample.csv", "Michael Phelps", "50m doggy paddle")` returns `(2, 62)` because Michael Phelps has two entries for the event with a rounded average time of 62 seconds. Note that `round(62.5) => 62`
- `get_swim_report("swimtimes-sample.csv", "Shanti Pereira", "50m backstroke")` returns `(1, 42)` because Shanti swam the 50m backstroke once in 42 seconds.
- `get_swim_report("swimtimes-sample.csv", "Shanti Pereira", "50m doggy paddle")` returns `(0, 0)` because Shanti did not swim the 50m doggy paddle.

Use `q2b_test.py` to test your code. Note that it also uses `swimtimes-sample2.csv` for some tests. Your `q2b.py` should contain the entire function definition of `get_swim_report`. We will grade only your `q2b.py` using a different test script.

Question 3 [4 marks]

Part A (Difficulty Level **/*)**

To manage its inventory proactively, XYZ Company is working to add a minimum stock number (`min_stock`) for each product. When the inventory level falls to or below this number, the company will order the product from its supplier.

In `q3a.py`, implement a function called `manage_inventory(filename)` that reads current inventory data. It takes one parameter:

- `filename` (type: `str`): The name of a text file containing all the current product inventory information.

The input text file consists of multiple lines of inventory records.

- `product_name`: the name of the product.
- `category`: the category of the product.
- `price_per_unit`: the product price XYZ company is selling at to buyers, in the format of `'$0.00'` or `'0.00'`.
- `cost_per_unit`: the product price company XYZ is buying at from suppliers, in the format of `'$0.00'` or `'0.00'`.
- `quantity_in_stock`: an integer number indicating the current stock of the product.

An example inventory text file named `'inventory1.txt'` is as follows. Note that the rows are not sorted in any order and each product appears only once in the text file. All strings are case-sensitive.

```
Laptop,Electronics,$1200.00,$1100.00,25
Phone,Electronics,800.00,$700.00,50
Chair,Furniture,$60.00,50.00,100
Sofa,Furniture,$960.00,$800.00,10
TV,Electronics,500.00,$400.00,30
Table,Furniture,$140.00,100.00,20
Headphones,Electronics,$150.00,$100.00,60
Refrigerator,Appliances,1200.00,$700.00,8
Microwave,Appliances,$200.00,$100.00,15
Lamp,Furniture,$40.00,$15.00,40
```

The function returns a **dictionary.** The keys are product names (type: `str`) and the corresponding values are lists, each of which contains information in the form of:

`[quantity_in_stock(type: int), min_stock(type: int)]`

The minimum stock number is calculated as follows:

$$\textbf{profit margin} = \frac{\textbf{price_per_unit} - \textbf{cost_per_unit}}{\textbf{cost_per_unit}}$$

Profit margin	Minimum stock number
>=0.5	3
>=0.2 and <0.5	8
<0.2	15

You can assume `cost_per_unit` is never zero.

The function will return an empty dictionary if the inventory file is empty.

For example:
`manage_inventory('inventory1.txt')`

should return:

```
{'Laptop': [25, 15], 'Phone': [50, 15], 'Chair': [100, 8], 'Sofa': [10, 8],
'TV': [30, 8], 'Table': [20, 8], 'Headphones': [60, 3], 'Refrigerator': [8,
3], 'Microwave': [15, 3], 'Lamp': [40, 3]}
```

Use `q3a_test.py` to test your code. Your `q3a.py` should contain the entire function definition of `manage_inventory`. We will grade only your `q3a.py` using a different test script.

Part B (Difficulty Level **/*)**

During a sales event, XYZ company needs to process customer orders by checking its inventory.

In `q3b.py`, implement a function called `process_orders(filename, list_of_orders)` that reads a text file and checks stock levels to process sales orders. Each time an order is processed, the quantity in stock should be updated accordingly.

This function takes in two parameters:

- `filename` (type: `str`): The name of the input text file containing all the inventory information of products, as in Part A.

- `list_of_orders` (type: `list[tuples]`): Each tuple in the list contains the information of an order in the form of: `(order_id` (type: `int`), `buyer` (type: `str`), `product_name` (type: `str`), `quantity_ordered` (type: `int`)). The orders are sorted by their unique `order_id`. All strings are case-sensitive.

For example:

```
list_of_orders1 = [(1, 'Alice', 'Laptop', 20), (2, 'Bob', 'Phone', 50), (3,
'Carol', 'Chair', 10), (4, 'David', 'TV', 30), (5, 'Alice', 'Headphones',
40), (6, 'Bob', 'Sofa', 10), (7, 'Carol', 'Microwave', 20), (8, 'David',
'Table', 10)]
```

The function returns a **dictionary.** The keys are buyers, and the corresponding values are lists of tuples.

Each tuple contains information in the form of:
`(order_id` (type: `int`), `product_name` (type: `str`), `quantity_ordered` (type: `int`), `quantity_filled` (type: `int`)).

`order_id`, `product_name`, and `quantity_ordered` are exactly as shown in `list_of_orders`.

The tuples must appear in the same order as in `list_of_orders`. Since different buyers can place orders for the same product, orders listed earlier in `list_of_orders` are fulfilled before orders that are listed later. Stock levels are updated after each order is processed.

The value of `quantity_filled` may be less than `quantity_ordered` if there is insufficient stock to fill the order in full.

An empty dictionary will be returned if `list_of_orders` is empty.

If the inventory file is empty, the `quantity_filled` should be 0 for all the orders. You can assume that products ordered by the buyers are all recorded in the inventory file if it is not empty.

For example:
`process_orders ('inventory1.txt', list_of_orders 1)` should return
`{'Alice': [(1, 'Laptop', 20, 20), (5, 'Headphones', 40, 40)], 'Bob': [(2, 'Phone', 50, 50), (6, 'Sofa', 10, 10)], 'Carol': [(3, 'Chair', 10, 10), (7, 'Microwave', 20, 15)], 'David': [(4, 'TV', 30, 30), (8, 'Table', 10, 10)]}`
because all orders in `list_of_orders1` can be fulfilled except for `Carol`'s order for the `Microwave`, where only 15 out of the requested 20 units could be provided based on the available stock in the inventory.

Hint: You may import and use `manage_inventory (filename)` defined in Part A. This is not required to solve the problem.

Use `q3b_test.py` to test your code. Your `q3b.py` should contain the entire function definition of `process_orders`. We will grade only your `q3b.py` using a different test script.

Question 4 [5 marks]

`ThinkCoin` is a popular cryptocurrency with a market value of over a trillion dollars.

As with physical currency, `ThinkCoin` coins have distinct values and are interchangeable.

For example, a user (represented by an address, see below) may have several coins of denominations `[5, 5, 1, 2]`. In a transaction that requires a `ThinkCoin` value of 9, the user must spend coins with value `[5, 5]` and receives change in the form of one coin with value 1. The user will now possess the following coins: `[1, 1, 2]`.

This problem models a sequence of `ThinkCoin` transactions.

Part A (Difficulty Level *)**
Implement a function called `get_invalid_transactions` in `q4a.py`.

This function accepts one parameter:

- `filename` (`type: str`): A text file containing a set of `ThinkCoin` transactions. The format of the file is defined below.

This function returns:

- A list of invalid transaction IDs (type: `list[str]`) in the same order as in the text file. See below for the definitions of valid/invalid transactions. If there are no invalid transactions, the function returns an empty list.

Text File Format
The file contains a simplified representation of how funds move through a cryptocurrency system, from initial mining (coinbase) to various users via transactions. The transactions are ordered: the 1st line in the file represents the 1st transaction and so on. Each transaction is represented in the following format:

```
TX_ID|IN:Address:Amount|…|OUT:Address:Amount|…
```

Example transactions from a text file named `transactions_1.txt` are shown below.

```
TX001|IN:COINBASE:9|OUT:MinerA:9
TX002|IN:COINBASE:11|OUT:MinerB:11
TX003|IN:COINBASE:11|OUT:MinerB:11
TX004|IN:COINBASE:14|OUT:MinerC:14
TX005|IN:COINBASE:7|OUT:MinerD:7
TX006|IN:MinerA:9|OUT:MinerA:1|OUT:Alice:8
TX007|IN:MinerB:11|OUT:MinerB:1|OUT:Bob:10
TX008|IN:MinerA:9|OUT:Isaac:8|OUT:MinerA:1
TX009|IN:MinerC:14|IN:MinerD:7|OUT:Charlie:20|OUT:MinerC:1
TX010|IN:COINBASE:9|OUT:MinerA:9
TX011|IN:Alice:8|IN:Bob:10|OUT:David:12|OUT:Alice:1|OUT:Charlie:5
TX012|IN:Charlie:20|OUT:Eve:15|OUT:Charlie:5
TX013|IN:Charlie:2|OUT:Eve:2
TX014|IN:Eve:2|OUT:Jack:2
TX015|IN:MinerA:1|IN:MinerA:1|IN:Alice:1|OUT:Frank:1|OUT:David:2
```

- `TX_ID` represents the transaction ID, which is a unique string of letters and digits. The order of the transactions is determined by the line sequence in the file and not by the `TX_ID`.
- `IN` represents inputs (sources of funds). `OUT` represents outputs (recipients of funds) which only appear after all inputs. Each transaction has at least one input and one output.
- `Amount` represents the value of a `ThinkCoin` coin in transactions. It is always a positive integer.
- `Address` is a unique non-empty string which identifies a particular `ThinkCoin` user from whom coins are withdrawn (`IN`) or to whom coins are deposited (`OUT`).
- For each transaction, the total input amount is always equal to the total output amount.
- If the address is `COINBASE`, for example in `TX001` - the transaction is considered a coinbase transaction. This introduces new coins into circulation. Coinbase transactions have only one input which is `IN:COINBASE`, and only one output. Coinbase transactions are always valid.
- Coins received by an address (`OUT`) are distinct from other coins received at that address. For example, after `TX003`, `MinerB` has two different coins with value 11, each (received from `TX002` and `TX003`).
- Coins received by an address (`OUT`) that have equal value can be used interchangeably.
    - `Charlie` obtains two coins of value 5 from `TX011` and `TX012` respectively. These two coins are distinct but can be used interchangeably.
- Coins must be spent (`IN`) completely in a transaction. For example, in `TX007`, `MinerB` spends one coin with value 11, as `MinerB` has two coins of value 11, each. `MinerB` receives (`OUT`) a new coin with value 1 as change, and a coin of value 10 goes to `Bob`. After `TX007`, `MinerB` possesses two coins with values 1 and 11, respectively.
- All strings in the file are case-sensitive. There are no spaces between the characters in each line. The keywords `COINBASE`, `IN`, and `OUT` are not used for any other purposes.

All transactions are valid unless they violate this rule: an address **cannot** spend coins (`IN`) it does not possess, with the sole exception of `COINBASE`, which is able to create new coins. Invalid transactions are not processed. More examples below:

- After `TX001`, `MinerA` has one coin with value 9, which is then used in `TX006`. Both are valid transactions.
- Transaction `TX008` is **invalid** as `MinerA` has only 1 coin with value 1 after `TX006`. So `TX008` was not processed.
- `TX013` is **invalid** as `Charlie` possesses only two coins of value 5 each, obtained in previous, valid transactions `TX011` and `TX012`. `Charlie` cannot spend a coin of value 2 as `Charlie` does not have it.
- `TX014` is also **invalid** as `Eve` does not possess a coin of value 2 since `TX013` was not processed (invalid).
- `TX015` is **invalid** as `MinerA` possesses just one coin with value 1, and this transaction requires two coins with value 1, each.

`get_invalid_transactions('transactions_1.txt')` should therefore return:
`['TX008', 'TX013', 'TX014', 'TX015']`

The transaction IDs in the returned list are ordered according to the order in the text file.

Use `q4a_test.py` to test your code. Your `q4a.py` should contain the entire function definition of `get_invalid_transactions`. We will only grade your `q4a.py` using a different test script.

Part B (Difficulty Level *)**

In `ThinkCoin`, there is the problem of selecting a combination of coins from an address to satisfy a given target amount while minimizing the amount of change.

The text file `transactions_2.txt` contains the following transaction sequence:

```
TX001|IN:COINBASE:10|OUT:MinerA:10
TX002|IN:COINBASE:11|OUT:MinerB:11
TX003|IN:COINBASE:14|OUT:MinerC:14
TX004|IN:COINBASE:7|OUT:MinerD:7
TX005|IN:MinerA:10|OUT:Alice:8|OUT:MinerA:2
TX006|IN:MinerB:11|OUT:Bob:10|OUT:MinerB:1
TX007|IN:MinerC:14|IN:MinerD:7|OUT:Charlie:20|OUT:MinerC:1
TX008|IN:COINBASE:2|OUT:MinerA:2
TX009|IN:Alice:8|IN:Bob:10|OUT:David:17|OUT:Alice:1
TX010|IN:Charlie:20|OUT:Eve:19|OUT:Charlie:1
TX011|IN:Eve:19|OUT:Jack:19
TX012|IN:COINBASE:4|OUT:MinerA:4
TX013|IN:COINBASE:8|OUT:MinerA:8
TX014|IN:Jack:19|OUT:MinerA:6|OUT:MinerA:2|OUT:MinerA:4|OUT:Jack:7
```

At the end of the above transaction sequence, `MinerA` possesses:

- Three coins of value 2: `[2, 2, 2]`
- Two coins of value 4: `[4, 4]`
- One coin of value 6: `[6]`
- One coin of value 8: `[8]`

There are many ways to select coins if the next transaction requires an input with a total value of at least 10:

- `[4, 6]`

- `[2, 4, 6]`
- `[4, 8]`
- `[2, 8]`
- `[2, 2, 8]`
- …

To address this problem, implement a function called `get_coins` in `q4b.py`.
This function accepts three parameters:

- `filename` (type: `str`): A text file containing a set of `ThinkCoin` transactions as described in Part A. We assume that all transactions in the file are valid. The selection of coins should be carried out after all transactions in the file have already been processed.
- `address` (type: `str`): An address with coins. We assume the address is valid and exists in the file. It is not a `COINBASE` address.
- `target` (type: `int`): The target amount which is a positive integer.

This function returns a list of tuples `[(coin1, coin2,…, coinN), …]`, in which each tuple contains the selected coins (which are positive integers) from the coins possessed by the `address`.

The returned list must contain **all** valid tuples. A tuple is considered valid if:

- The sum of the coin values in each tuple must be at least the same as the target amount `target`, **and**
- The difference, if any, between the sum of the coin values in each tuple and the target amount, `target`, is the smallest possible (to minimize the change), **and**
- All tuples in the returned list must be unique.

Notes:

- The order of the coins in each tuple does not matter, i.e., `(1,2,3)` is considered to be the same as `(3,2,1)` so only one of these two should be included in the returned list.
- There may be distinct coins of the same value in each tuple, e.g., `(3,2,2,3,3)`.
- The order of the tuples in the list does not matter.
- The function returns an empty list if there are no coins from `address` that satisfy `target`.

For example:

- `get_coins('transactions_2.txt', 'MinerA', 10 )` returns:
  `[(2, 2, 2, 4), (2, 2, 6), (2, 4, 4), (2, 8), (4, 6)]`
  After all transactions in the file have been processed, `MinerA` possesses coins with the following values: `[2, 2, 4, 8, 6, 2, 4]`. The output list comprises all possible coin combinations from `MinerA` that exactly match the target amount of 10, resulting in the smallest change of 0.
- `get_coins('transactions_2.txt', 'MinerA', 13 )` returns:
  `[(2, 2, 2, 4, 4), (2, 2, 2, 8), (2, 2, 4, 6), (2, 4, 8), (4, 4, 6), (6, 8)]`. The output list comprises all possible coin combinations from `MinerA` that result in the smallest change of 1.
- `get_coins('transactions_2.txt', 'MinerB', 5 )` returns: `[]`
  `MinerB` only has one coin of value 1 after all transactions in the file are processed.

Hint: You can import code from your own `q4a.py` if necessary. You can also make use of the `combinations` function from the `itertools` package in Python. Try the code below to figure out how to use it:

```
from itertools import combinations
my_list = ['a', 'b', 'c', 'd']
size = 3
for combo in combinations(my_list, size):
    print(combo)
```

Use `q4b_test.py` to test your code. Your `q4b.py` should contain the entire function definition of `get_coins`. We will only grade your `q4b.py` using a different test script.

END OF PAPER